asio socket同步读写

Posted on 2023-02-14 15:13:15 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2LifP1RzCnDxtbaepFJkHVdr2ce)

同步写write_some

boost::asio提供了几种同步写的api，write_some可以每次向指定的空间写入固定的字节数，如果写缓冲区满了，就只写一部分，返回写入的字节数。

```
1. void  wirte_to_socket(asio::ip::tcp::socket& sock) {
2.     std::string buf = "Hello World!";
3.     std::size_t  total_bytes_written = 0;
4.     //循环发送
5.     //write_some返回每次写入的字节数
6.     //total_bytes_written是已经发送的字节数。
7.     //每次发送buf.length()- total_bytes_written)字节数据
8.
9.     while (total_bytes_written != buf.length()) {
10.        total_bytes_written += sock.write_some(
11.            asio::buffer(buf.c_str()+total_bytes_written,
12.                buf.length()- total_bytes_written));
13.    }
14. }
```

同步写send

write_some使用起来比较麻烦，需要多次调用，asio提供了send函数。send函数会一次性将buffer中的内容发送给对端，如果有部分字节因为发送缓冲区满无法发送，则阻塞等待，直到发送缓冲区可用，则继续发送完成。

```
int send_data_by_send(){
    std::string raw_ip_address = "127.0.0.1";
    unsigned short port_num = 3333;

    try {
        asio::ip::tcp::endpoint
            ep(asio::ip::address::from_string(raw_ip_address),
                port_num);

        asio::io_service ios;

        // Step 1. Allocating and opening the socket.
        asio::ip::tcp::socket sock(ios, ep.protocol());

        sock.connect(ep);
        std::string buf = "Hello World!";
        int send_length = sock.send(asio::buffer(buf.c_str(), buf.length()));
        if (send_length <= 0) {
            cout << "send failed" << endl;
            return 0;
        }
    }
    catch (system::system_error& e) {
        std::cout << "Error occured! Error code = " << e.code()
            << ". Message: " << e.what();

        return e.code().value();
    }
    return 0;
}
```

同步写write

类似send方法，asio还提供了一个write函数，可以一次性将所有数据发送给对端，如果发送缓冲区满了则阻塞，直到发送缓冲区可用，将数据发送完成。

```
1. int send_data_by_wirte() {
2.     std::string raw_ip_address = "127.0.0.1";
3.     unsigned short port_num = 3333;
4.
5.     try {
6.         asio::ip::tcp::endpoint
7.             ep(asio::ip::address::from_string(raw_ip_address),
8.                 port_num);
9.
10.        asio::io_service ios;
11.
12.        // Step 1. Allocating and opening the socket.
13.        asio::ip::tcp::socket sock(ios, ep.protocol());
14.
15.        sock.connect(ep);
16.        std::string buf = "Hello World!";
17.        int send_length  = asio::write(sock, asio::buffer(buf.c_str(), buf.length()));
18.        if (send_length <= 0) {
19.            cout << "send failed" << endl;
20.            return 0;
21.        }
22.    }
23.    catch (system::system_error& e) {
24.        std::cout << "Error occured! Error code = " << e.code()
25.            << ". Message: " << e.what();
26.
27.        return e.code().value();
28.    }
29.    return 0;
30. }
```

同步读read_some

同步读和同步写类似，提供了读取指定字节数的接口read_some

```
std::string read_from_socket(asio::ip::tcp::socket& sock) {
    const unsigned char MESSAGE_SIZE = 7;
    char buf[MESSAGE_SIZE];
    std::size_t total_bytes_read = 0;

    while (total_bytes_read != MESSAGE_SIZE) {
        total_bytes_read += sock.read_some(
            asio::buffer(buf + total_bytes_read,
                MESSAGE_SIZE - total_bytes_read));
    }

    return std::string(buf, total_bytes_read);
}


int read_data_by_read_some() {
    std::string raw_ip_address = "127.0.0.1";
    unsigned short port_num = 3333;

    try {
        asio::ip::tcp::endpoint
            ep(asio::ip::address::from_string(raw_ip_address),
                port_num);

        asio::io_service ios;

        asio::ip::tcp::socket sock(ios, ep.protocol());

        sock.connect(ep);

        read_from_socket(sock);
    }
    catch (system::system_error& e) {
        std::cout << "Error occured! Error code = " << e.code()
            << ". Message: " << e.what();

        return e.code().value();
    }

    return 0;
}
```

同步读receive

可以一次性同步接收对方发送的数据

```
1. int read_data_by_receive() {
2.     std::string raw_ip_address = "127.0.0.1";
3.     unsigned short port_num = 3333;
4.
5.     try {
6.         asio::ip::tcp::endpoint
7.             ep(asio::ip::address::from_string(raw_ip_address),
8.                 port_num);
9.
10.        asio::io_service ios;
11.
12.        asio::ip::tcp::socket sock(ios, ep.protocol());
13.
14.        sock.connect(ep);
15.        const unsigned char BUFF_SIZE = 7;
16.         char buffer_receive[BUFF_SIZE];
17.        int receive_length =  sock.receive(asio::buffer(buffer_receive, BUFF_SIZE));
18.        if (receive_length <= 0) {
19.            cout << "receive failed" << endl;
20.        }
21.    }
22.    catch (system::system_error& e) {
23.        std::cout << "Error occured! Error code = " << e.code()
24.            << ". Message: " << e.what();
25.
26.        return e.code().value();
27.    }
28.
29.    return 0;
30. }
```

同步读read

可以一次性同步读取对方发送的数据

```
int read_data_by_read() {
    std::string raw_ip_address = "127.0.0.1";
    unsigned short port_num = 3333;

    try {
        asio::ip::tcp::endpoint
            ep(asio::ip::address::from_string(raw_ip_address),
                port_num);

        asio::io_service ios;

        asio::ip::tcp::socket sock(ios, ep.protocol());

        sock.connect(ep);
        const unsigned char BUFF_SIZE = 7;
        char buffer_receive[BUFF_SIZE];
        int receive_length = asio::read(sock, asio::buffer(buffer_receive, BUFF_SIZE));
        if (receive_length <= 0) {
            cout << "receive failed" << endl;
        }
    }
    catch (system::system_error& e) {
        std::cout << "Error occured! Error code = " << e.code()
            << ". Message: " << e.what();

        return e.code().value();
    }

    return 0;
}
```

读取直到指定字符

我们可以一直读取，直到读取指定字符结束

```
std::string  read_data_by_until(asio::ip::tcp::socket& sock) {
    asio::streambuf buf;

    // Synchronously read data from the socket until
    // '\n' symbol is encountered.
    asio::read_until(sock, buf, '\n');

    std::string message;

    // Because buffer 'buf' may contain some other data
    // after '\n' symbol, we have to parse the buffer and
    // extract only symbols before the delimiter.

    std::istream input_stream(&buf);
    std::getline(input_stream, message);
    return message;
}
```

源码链接

https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(4428) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(596) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)